2.5. Mecanismos de decisión (Criterio 2.e)

- **Resultado de Aprendizaje 2 (RA2):** *Escribe sentencias simples, aplicando la sintaxis del lenguaje y verificando su ejecución sobre navegadores Web.*

- **Criterio Curricular Oficial: CE 2.e** (*Se han utilizado mecanismos de decisión en la creación de bloques de sentencias*).

- **Ponderación del Criterio:** (**20% del RA2** | **1% sobre la calificación final del módulo**).

A. ¿Qué es un Mecanismo de Decisión y qué es un Bloque?

Hasta ahora, las instrucciones escritas se ejecutaban de forma lineal: una tras otra, de arriba hacia abajo. Los **mecanismos de decisión** permiten alterar el flujo secuencial de un programa: evalúan una condición booleana (true o false) y deciden qué camino de código ejecutar y cuál ignorar.

Un **bloque** es un grupo de cero o más sentencias agrupadas dentro de un par de llaves { ... }. En JavaScript moderno, los bloques no solo agrupan instrucciones, sino que establecen una barrera de ámbito (*block scope*) para las variables let y const:

```
// Bloque delimitado por llaves
{
  let datoLocal = "Visible solo dentro de estas llaves";
  console.log(datoLocal);
}
// console.log(datoLocal); // Lanza ReferenceError
```

B. La Estructura Condicional if...else

1. Estructura Simple y Doble

Evalúa una expresión entre paréntesis. Si el resultado es verdadero (*truthy*), se ejecuta el bloque asociado al if; si es falso (*falsy*), se ejecuta el bloque else (si existe):

```
let saldoCuenta = 50;
let costeArticulo = 30;

// Estructura if...else básica
if (saldoCuenta >= costeArticulo) {
  saldoCuenta -= costeArticulo;
  console.log(`Compra completada. Saldo restante: ${saldoCuenta}
€`);
} else {
  console.log("Error: Fondos insuficientes para realizar la
compra.");
```

```
}
```

2. Omisión Opcional de Llaves en Sentencias Únicas

Si un bloque if o else contiene una única instrucción, el estándar permite omitir las llaves {}:

```
let autenticado = true;

// Válido sintácticamente en JavaScript:
if (autenticado)
  console.log("Acceso concedido");
else
  console.log("Acceso denegado");
```

Aunque la sintaxis lo autorice, **omitir las llaves se considera una mala práctica**. Si un programador añade posteriormente una segunda línea pensando que forma parte de la condición, esa segunda línea se ejecutará siempre de forma incondicional, introduciendo fallos críticos de seguridad.

C. Decisiones Múltiples: if...else if...else y Anidamiento

1. Encadenamiento Secuencial

Permite comprobar una cadena ordenada de condiciones. En el momento en que una de ellas se evalúa a true, se ejecuta su bloque y el motor salta directamente fuera de toda la estructura, ignorando las demás ramas:

```
let notaExamen = 7.5;

if (notaExamen > 0 && notaExamen < 5) {
  console.log("Suspenso");
} else if (notaExamen < 7) {
  console.log("Aprobado");
} else if (notaExamen < 9) {
  console.log("Notable");
} else if (notaExamen <= 10) {
  console.log("Sobresaliente");
} else {
  console.log("Error: Calificación fuera del rango legal
(0-10).");
}
```

2. Estructuras Condicionales Anidadas

Consiste en introducir un bloque if dentro de otro bloque if cuando una decisión secundaria depende obligatoriamente del éxito de una decisión previa:

```
let tieneEntrada = true;
let edad = 16;
```

```
if (tieneEntrada) {
  // Decisión secundaria dependiente
  if (edad >= 18) {
    console.log("Pase general autorizado.");
  } else {
    console.log("Acceso autorizado con acreditación de tutor
legal.");
  }
} else {
  console.log("Acceso denegado: No posee entrada válida.");
}
```

D. La Sentencia de Selección Múltiple: switch...case...default

La estructura switch es idónea cuando una misma variable o expresión se compara contra múltiples valores discretos conocidos.

```
let metodoPago = "BIZUM";

switch (metodoPago) {
  case "TARJETA":
    console.log("Pasarela bancaria seleccionada: Redsys");
    break; // Impide continuar hacia abajo

  case "PAYPAL":
    console.log("Redirigiendo a la API segura de PayPal");
    break;

  case "BIZUM":
    console.log("Solicitando número de teléfono móvil");
    break;

  default:
    console.log("Método de pago no compatible.");
    break;
}
```

1. Regla Crítica: Igualdad Estricta (===) en switch

El motor compara la expresión contra cada etiqueta case aplicando **estrictamente ===**. Esto significa que **no hay conversión automática de tipos (coerción)**:

```
let nivelAcceso = "1"; // Tipo String

switch (nivelAcceso) {
  case 1: // Tipo Number
    console.log("Nivel Administrador");
    break;
  default:
```

```
    console.log("No coincide porque '1' !== 1 (Tipos
diferentes)");
    break;
}
// Salida: No coincide porque '1' !== 1 (Tipos diferentes)
```

2. El Efecto *Fall-Through* (Caída Libre Secuencial)

Si se omite intencionadamente la instrucción break, el flujo de ejecución "cae" y ejecuta las instrucciones de los siguientes casos sin comprobar si coinciden o no, hasta toparse con un break. Se utiliza de forma productiva para agrupar casos que comparten la misma lógica:

```
let mes = "ABRIL";

switch (mes) {
  // Casos agrupados aprovechando el Fall-Through:
  case "DICIEMBRE":
  case "ENERO":
  case "FEBRERO":
    console.log("Estación: Invierno");
    break;

  case "MARZO":
  case "ABRIL":
  case "MAYO":
    console.log("Estación: Primavera");
    break;

  default:
    console.log("Otra estación del año");
    break;
}
```

Posición libre de `default`: La cláusula `default` no tiene por qué situarse obligatoriamente como el último bloque dentro de las llaves del `switch`. Puede ubicarse al principio o en una posición intermedia entre los distintos `case`.

Obligatoriedad de `break` en `default` intermedio: Cuando `default` se ubica al final de la estructura, la sentencia `break` es técnicamente prescindible (aunque recomendada por estilo) porque el flujo llega al cierre de la llave `}`. Sin embargo, **si `default` se coloca al principio o en medio, debe llevar obligatoriamente su propia sentencia `break`**. De lo contrario, si la expresión no coincide con ningún `case` previo, el flujo entrará en `default` y continuará ejecutando en cascada (*fall-through*) todos los `case` que se encuentren por debajo:

```
var valor = 99;

// default situado al inicio de la estructura
```

```
switch (valor) {
  default:
    console.log("Valor no reconocido");
    break; // OBLIGATORIO: Si se omite, entra aquí y continúa
ejecutando case 1 y case 2
  case 1:
    console.log("Uno");
    break;
  case 2:
    console.log("Dos");
    break;
}
```

E. Expresión Condicional Ternaria (? :)

El operador ternario es el único operador en JavaScript que toma tres operandos. Permite condensar una decisión simple en una **expresión que devuelve un valor**:

$$condicion\ ?\ valorSiVerdadero : valorSiFalso$$

```
let edad = 20;

// Forma clásica con if...else (4 líneas):
let estadoA;
if (edad >= 18) {
  estadoA = "Adulto";
} else {
  estadoA = "Menor";
}

// Forma compacta con operador ternario (1 línea):
const estadoB = (edad >= 18) ? "Adulto" : "Menor";
console.log(estadoB); // "Adulto"

// Inserción directa dentro de plantillas literales:
console.log(`El usuario es ${edad >= 18 ? "mayor" : "menor"} de
edad.`);
```

F. Decisiones Mediante Operadores de Cortocircuito

Los operadores lógicos (`&&`, `||`, `??`) se usan frecuentemente en JavaScript como atajos para ejecutar funciones de manera condicional. Esto se conoce como *evaluación de circuito corto* (*short-circuit evaluation*).

1. El operador `&&` (Ejecución Condicional)

El operador `&&` evalúa de izquierda a derecha. Ejecuta el lado derecho **solo si el lado izquierdo es verdadero (`true` o *truthy*)**. Si el operando izquierdo es falso, el motor no evalúa la derecha. Si es verdadero, evalúa y retorna la expresión derecha:

- **Cuándo usarlo:** Cuando quieres ejecutar una función solo si una variable existe o una condición se cumple (equivalente a un `if` corto).
- **Regla:** `SI_ESTO_ES_VERDADERO && EJECUTA_ESTA_FUNCION()`

```
const usuarioLogueado = true;
const abrirPanel = () => console.log("Panel abierto");

// Como es true, la función se ejecuta
usuarioLogueado && abrirPanel();
```

2. El operador `||` (Ejecución de Respaldo / Fallback)

El operador `||` se detiene en el primer valor verdadero. Ejecuta el lado derecho **solo si el lado izquierdo es falso (`false` o *falsy*: `null`, `undefined`, `0`, `""`, `NaN`)**.

- **Cuándo usarlo:** Cuando quieres ejecutar una función de respaldo si la primera opción no es válida o falló.
- **Regla:** `SI_ESTO_FALLA || EJECUTA_ESTA_OTRA_FUNCION()`

```
const obtenerConfiguracionLocal = () => null; // Simulamos
que no hay datos
const descargarConfiguracionNube = () =>
console.log("Descargando desde la nube...");

// Como la primera devuelve null (falsy), se ejecuta la
segunda
obtenerConfiguracionLocal() || descargarConfiguracionNube();
```

3. El operador `??` (Operador Nullish Coalescing)

El operador `??` es más estricto que `||`. Ejecuta el lado derecho **solo si el lado izquierdo es estrictamente `null` o `undefined`**. Si el lado izquierdo es `0` o una cadena vacía `""`, no ejecutará la función.

- **Cuándo usarlo:** Cuando valores legítimos como el número `0` o un texto vacío `""` deben ser respetados, y solo quieres un respaldo ante la ausencia total de datos.
- **Regla:** `SI_ESTO_ES_NULL_O_UNDEFINED ?? EJECUTA_ESTA_FUNCION()`

```
const puntajeActual = 0; // El jugador tiene cero puntos
(valor válido)
const inicializarMarcador = () => console.log("Marcador
reseteado a 10");

// Con || se habría ejecutado porque 0 es falsy.
```

```
// Con ?? NO se ejecuta porque 0 NO es null ni undefined.
puntajeActual ?? inicializarMarcador();
```

G. Reducción de Complejidad Ciclomática mediante Cláusulas de Guarda (*Guard Clauses*) y Retorno Temprano (*Early Return*)

En consonancia con las exigencias de la industria del desarrollo de software actual y los análisis estáticos en entornos de integración continua (CI/CD mediante *ESLint*, reglas de *SonarQube* y complejidad ciclomática de McCabe), se exige evitar el anidamiento piramidal de estructuras condicionales (*Arrow Anti-Pattern* / código espagueti).

La **complejidad ciclomática** (desarrollada por Thomas J. McCabe en 1976) es una métrica de software que mide la **cantidad de caminos lógicos independientes** que puede tomar el flujo de ejecución de un programa o función.

En términos sencillos: mide cuántos caminos distintos existen dentro de una función y, por tanto, **cuántas pruebas unitarias como mínimo se necesitan** para cubrir todos los posibles escenarios.

¿Por qué se penaliza una complejidad alta?

1. **Mayor probabilidad de bugs:** Cuantas más bifurcaciones anidadas tiene una función, más difícil resulta para el programador prever todos los estados posibles.
2. **Dificultad en pruebas unitarias:** Una función con complejidad 15 requiere al menos 15 tests unitarios bien diseñados solo para tocar cada rama una vez.
3. **Mala legibilidad (deuda técnica):** Genera el antipatrón de la pirámide/flecha (*Arrow Anti-Pattern*), obligando a quien lee el código a retener mentalmente múltiples niveles de contexto condicional.

La regla de cálculo directo es:

$$ComplejidadCiclomática = 1 + Númerodepuntosdedecisión$$

Ejemplo práctico

Caso A: Función lineal (Complejidad = 1)

```
function duplicar(numero) {

    return numero * 2;
```

```
}
```

- **Puntos de decisión:** 0.
- **Complejidad:** $1 + 0 = 1$. Solo existe un camino posible.

Caso B: Función con bifurcaciones (Complejidad = 4)

```
function clasificarEdad(edad, tienePermiso) {
    if (edad >= 18) {            // +1 (decisión 1)
        return "Adulto";
    } else if (edad >= 16 && tienePermiso) { // +2 (decisión 2: else if, decisión 3: operador &&)
        return "Joven con permiso";
    }
    return "Menor";
}
```

- **Base:** 1
- **Puntos de decisión:** if (+1), else if (+1), && (+1).
- **Complejidad ciclomática:** $1 + 3 = 4$. Se necesitan al menos 4 casos de prueba distintos para garantizar cobertura total.

1. Principio Metodológico Aplicado

- **Aislamiento de precondiciones y errores:** Toda función debe validar sus argumentos, estados inválidos y casos de excepción al inicio del bloque. Si una condición previa no se cumple, la función debe abortar su flujo inmediatamente devolviendo un valor por defecto o lanzando una excepción (`return;`, `return false;`, `throw new Error(...)`).
- **Camino feliz despejado (*Happy Path*):** El flujo principal de ejecución de la función se mantendrá siempre con el nivel mínimo de indentación,

2. Ejemplo Práctico

```
// Mala práctica: anidamiento excesivo
function procesarRegistroUsuario(datos) {
    if (datos !== null && datos !== undefined) {
        if (datos.edad >= 18) {
            if (datos.email.includes("@")) {
                const usuario = { id: Date.now(), ...datos };
```

```
            guardarEnBaseDatos(usuario);
            return { exito: true, usuario };
        } else {
            return { exito: false, error: "Formato de email incorrecto" };
        }
    } else {
        return { exito: false, error: "El usuario debe ser mayor de edad" };
    }
  } else {
    return { exito: false, error: "Datos no proporcionados" };
  }
}


// Buena práctica: complejidad ciclomática reducida: Cláusulas de guarda y retorno temprano
function procesarRegistroUsuario(datos) {
  // 1. Cláusulas de guarda (validación de precondiciones al inicio)
  if (!datos) {
    return { exito: false, error: "Datos no proporcionados" };
  }
  if (datos.edad < 18) {
    return { exito: false, error: "El usuario debe ser mayor de edad" };
  }
  if (!datos.email || !datos.email.includes("@")) {
    return { exito: false, error: "Formato de email incorrecto" };
  }

  // 2. Camino principal (Happy Path) con sangría base
  const usuario = { id: Date.now(), ...datos };
  guardarEnBaseDatos(usuario);
  return { exito: true, usuario };
}
```

H. Ejercicios.

Ejercicio 1. Refactorización y Control de Complejidad Ciclomática

En entornos de integración continua (CI/CD) de empresas de desarrollo frontend, el código fuente pasa por linters y analizadores estáticos automáticos (como **ESLint** y **SonarQube**) antes de integrarse en la rama principal.

El siguiente fragmento de código ha sido rechazado por el analizador estático con la siguiente advertencia:

```
[warning/error] complexity: Function 'calcularMatricula'
has a cyclomatic complexity of 8 (max allowed: 5).

[warning] max-depth: Blocks are nested 4 levels deep (max
allowed: 2).

[warning] no-else-return: Unnecessary 'else' after
'return'.
```

Código Fuente a Refactorizar (Antipatrón)

Guarda este código en un archivo llamado `matricula.js`:

```
// ANTIPATRÓN: Alta complejidad ciclomática y ramas 'else' innecesarias
function calcularMatricula(datos) {
    let resultado;

    if (datos !== null && typeof datos === "object") {
        if (typeof datos.precioBase === "number" && datos.precioBase > 0) {
            if (["general", "desempleado", "socio"].includes(datos.tipoEstudiante)) {
                let descuentoPorcentaje = 0;

                if (datos.tipoEstudiante === "desempleado") {
                    descuentoPorcentaje = 0.30;
                } else {
                    if (datos.tipoEstudiante === "socio") {
                        descuentoPorcentaje = 0.15;
                    }
                }

                let total = datos.precioBase * (1 - descuentoPorcentaje);

                if (datos.codigoDescuento === "WEB2026") {
                    total = total - 10;
                }

                if (total < 0) {
                    total = 0;
                }

                resultado = { valido: true, total: total };
            } else {
                resultado = { valido: false, error: "Tipo de estudiante no reconocido." };
            }
        } else {
            resultado = { valido: false, error: "El precio base debe ser un número positivo." };
        }
```

```
    } else {
        resultado = { valido: false, error: "Parámetros de matrícula inválidos." };
    }

    return resultado;
}
```

Objetivos de la Tarea

Debes reescribir la función `calcularMatricula(datos)` cumpliendo estrictamente con los siguientes requisitos:

1. **Aplicar Cláusulas de Guarda (*Guard Clauses*):** Valida los errores y precondiciones al principio de la función. Si los datos son inválidos, la función debe terminar inmediatamente con un `return` temprano (*Early Return*).
2. **Eliminar el anidamiento piramidal (*Arrow Anti-Pattern*):** El nivel máximo de indentación permitido dentro de la función será de **1 nivel** (ningún `if` anidado dentro de otro `if`).
3. **Prohibir ramas `else` innecesarias:** Tras una instrucción `return` anticipada, no debe usarse ninguna cláusula `else`.
4. **Camino Principal Despejado (*Happy Path*):** El cálculo exitoso y la devolución del importe final deben situarse al final del cuerpo principal de la función, sin estar encerrados en estructuras condicionales.
5. **Preservar el comportamiento exacto:** La función debe devolver exactamente los mismos objetos `{ valido: false, error: "..." }` y `{ valido: true, total: ... }` para las mismas entradas.

Banco de Pruebas de Verificación (Test Cases)

Añade las siguientes comprobaciones al final de tu archivo para verificar que tu función refactorizada produce idénticos resultados y supera todas las aserciones:

```
// Batería de pruebas: Comprobación de equivalencia funcional
console.assert(
    calcularMatricula(null).valido === false &&
    calcularMatricula(null).error === "Parámetros de matrícula inválidos.",
    "Fallo en Test 1: Manejo de entrada null"
);

console.assert(
    calcularMatricula({ precioBase: -20, tipoEstudiante: "general" }).valido === false &&
    calcularMatricula({ precioBase: -20, tipoEstudiante: "general" }).error === "El precio base debe ser un número positivo.",
    "Fallo en Test 2: Precio base negativo o cero"
);
```

```
console.assert(
    calcularMatricula({ precioBase: 100, tipoEstudiante: "extranjero" }).valido === false &&
    calcularMatricula({ precioBase: 100, tipoEstudiante: "extranjero" }).error === "Tipo de
estudiante no reconocido.",
    "Fallo en Test 3: Tipo de estudiante desconocido"
);

console.assert(
    calcularMatricula({ precioBase: 100, tipoEstudiante: "desempleado", codigoDescuento:
"WEB2026" }).total === 60,
    "Fallo en Test 4: Descuento desempleado (30%) + cupón (-10 €)"
);

console.assert(
    calcularMatricula({ precioBase: 100, tipoEstudiante: "socio" }).total === 85,
    "Fallo en Test 5: Descuento socio (15%) sin cupón"
);

console.assert(
    calcularMatricula({ precioBase: 5, tipoEstudiante: "general", codigoDescuento:
"WEB2026" }).total === 0,
    "Fallo en Test 6: Suelo mínimo a cero cuando el cupón excede el precio"
);

console.log("¡Todas las pruebas unitarias se ejecutaron con éxito!");
```

C: > Users > juanr > Documents > Desarrollo Cliente > Unidad 2_ Sintaxis del Lenguaje JavaScript > 2.5. Mecanismos de decisión > JS matricula.js > calcularMatricula

```javascript
function calcularMatricula(datos) {
    if (datos === null || typeof datos !== "object") {
        return { valido: false, error: "Parámetros de matrícula inválidos." };
    }

    if (typeof datos.precioBase !== "number" || !(datos.precioBase > 0)) {
        return { valido: false, error: "El precio base debe ser un número positivo." };
    }

    if (!["general", "desempleado", "socio"].includes(datos.tipoEstudiante)) {
        return { valido: false, error: "Tipo de estudiante no reconocido." };
    }

    let descuentoPorcentaje = 0;
    if (datos.tipoEstudiante === "desempleado") {
        descuentoPorcentaje = 0.30;
    }
    if (datos.tipoEstudiante === "socio") {
        descuentoPorcentaje = 0.15;
    }

    let total = datos.precioBase * (1 - descuentoPorcentaje);

    if (datos.codigoDescuento === "WEB2026") {
        total = total - 10;
    }

    if (total < 0) {
        total = 0;
    }

    return { valido: true, total: total };
```

Problems Output Debug Console Terminal Ports

```
C:\Program Files\nodejs\node.exe .\matricula.js
¡Todas las pruebas unitarias se ejecutaron con éxito!
¡Todas las pruebas unitarias se ejecutaron con éxito!
```